Aircraft Haptic Feedback Controller

Assignment 2 | ENGE X200 Microcontroller Programming

Hiten Gupta | a1918705 | Adelaide University

1 Introduction

This project implements an aircraft attitude feedback controller using the STM32C031C6 and Wokwi. An MPU6050 provides acceleration measurements, and three servos show feedback for roll, pitch and combined tilt. A pushbutton switches between manual and autopilot modes, and separate LEDs show the active mode. Finite state machines separate free, compliant and clamped operation and keep each state within its hysteresis band.

The code keeps register configuration, sensor acquisition, angle calculation and state decisions separate. I tested the 18 cases recorded in the results table in Wokwi. They cover the required individual-axis and tilt-only compliant examples, clamping, mode selection and selected return thresholds. Behaviour that still needs verification is covered in the limitations section.

2 Hardware and timing

The roll, pitch and tilt servos use PB4, PB5 and PB0, connected to TIM3 channels 1, 2 and 3 through alternate function 1. PA2 drives the autopilot LED and PB11 drives the manual LED. The button is on PA12 with a pull-down resistor. PB8 and PB9 carry the I2C clock and data through alternate function 6, with open-drain outputs and pull-ups. These assignments follow the brief [1].

The init function enables the peripheral clocks before it configures the pins, timers and I2C. GPIO_InitPin updates MODER and PUPDR, and GPIO_ConnectPeripheral selects the AFR field. PWM_Init selects PWM mode 1, enables each channel and enables compare preload. PWM_set_Pulse then writes the pulse to the matching TIM3 compare register.

Assuming a 48 MHz timer clock, PSC = 47 gives a one-microsecond TIM3 counter interval, and ARR = 19999 gives a 20 ms period, or 50 Hz. The compare registers therefore hold the pulse duration directly in microseconds. The conversion helper uses pulse = 500 + angle × 2000 / 180, rounds to an integer and limits the servo angle to 0–180°. Neutral roll and pitch use 1500 microseconds, and neutral tilt uses 500 microseconds. Compare preload applies new values at update events.

TIM16 uses PSC = 47999 as the millisecond timebase for communication timeouts and starts before the first I2C operation. I2C_Master_Init uses timing parameters 11, 19 and 19 from the lecture example [3]. MPU6050_Init wakes the sensor and selects its ±2g acceleration range. I assumed the clock configuration, and a fresh firmware build is still part of the final execution checks.

3 Sensor processing and I2C recovery

MPU6050_ReadAccel reads six consecutive bytes and reconstructs three signed 16-bit measurements. readAngles passes them to calc_roll, calc_pitch and calc_tilt in helper/helper.c. The equations are roll = atan2(ay, az), pitch = atan2(−ax, $\sqrt{(ay^2 + az^2)}$) and tilt = $\sqrt{(roll^2 + pitch^2)}$. They use atan2f and sqrtf, and the angles are converted from radians to degrees before the combined tilt is evaluated [1].

Raw acceleration values are enough to find orientation because a common scale cancels in the ratios. Test triples need a magnitude $\sqrt{(ax^2 + ay^2 + az^2)}$ of approximately 1g, which is the condition for constant-velocity cruise.

During development I hit I2C failures where an unsuccessful transaction returned false without clearing the peripheral's error state. I added a recovery sequence before every accelerometer read. It writes bits 4, 5, 8 and 9 of ICR to clear the selected status flags, clears CR1 bit 0 to disable the peripheral, then sets it again before requesting six acceleration bytes. This resets the peripheral before the next attempt, instead of clearing flags only after a successful transfer.

This fixed the communication problem I saw, but it is not a complete sensor-fault strategy. If a read still fails, readAngles returns false and keeps the previous angles. The interrupt currently ignores that return value, so later decisions use the previous successful sample. A future improvement would detect stale data explicitly and define the required fault response.

4 State machine design

Four state variables implement three diagram types. Pitch and roll each have an independent FREE, COMPLIANT and CLAMPED state, held in fsm_pitch and fsm_roll in main.c. Tilt uses fsm_tilt and the operating mode uses fsm_mode. Figure 1 shows the axis behaviour shared by roll and pitch, and Figures 2 and 3 show tilt and mode. All attitude states start as FREE and the mode starts as MANUAL.

For an individual axis, FREE commands 90°. At an absolute angle of at least 30° the state changes to COMPLIANT, which commands 90° plus the signed measured angle. From COMPLIANT, a magnitude above 50° enters CLAMPED. Positive clamping commands 140° and negative clamping commands 40°, which is ±50° around neutral.

The return thresholds give a fixed 5° hysteresis band. COMPLIANT returns to FREE below 25°, and CLAMPED returns to COMPLIANT at or below 45°. For example, if compliance is entered at 32°, the angle must still fall below 25° before the axis returns to FREE. Exactly 25° stays COMPLIANT in this implementation.

Tilt is never negative and starts with a 0° servo command. FREE changes to COMPLIANT above 20°, and the compliant output is twice the detected tilt. At 70° or above, COMPLIANT changes to CLAMPED and commands 140°. The return thresholds are below 15° for FREE and at or below 65° for COMPLIANT. These comparisons document the implementation, including the equality cases the brief's interval notation leaves out.

TIM3_IRQHandler reads the acceleration and updates pitch, roll, tilt and then mode on each timer update, so mode sees the latest axis states. MANUAL changes to AUTO when both axes are CLAMPED or a rising button edge is detected. AUTO changes to MANUAL only on a new rising edge while both axis states are FREE. The two LEDs are driven complementarily.

I considered an external interrupt for the button but chose to sample it inside fsm_mode. That keeps mode decisions in the same sequence as the angle-state updates and avoids a separate button interrupt modifying shared state. A saved previous button value detects a rising edge, so a held button cannot repeatedly toggle the mode. The trade-off is that presses must be sampled during the nominal 20 ms update cycle. Edge detection also does not give full mechanical debounce.

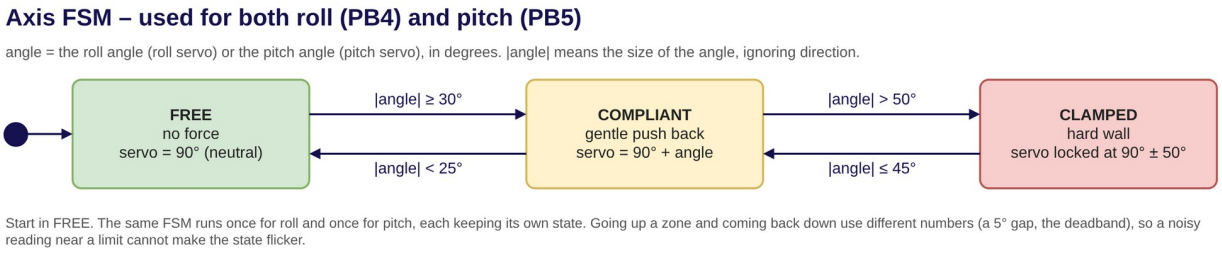


Figure 1 Axis FSM used independently for roll and pitch

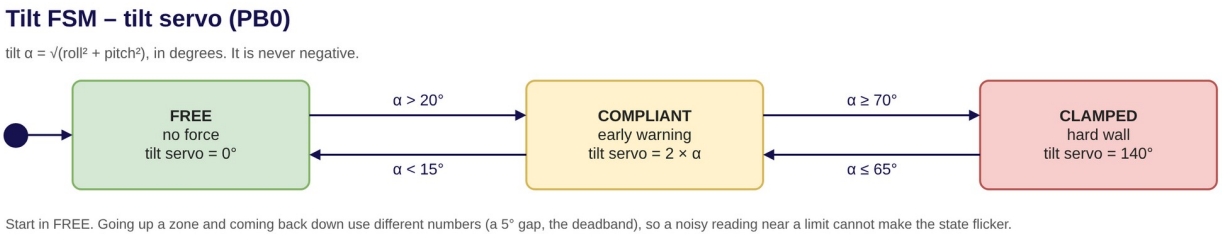


Figure 2 Unified tilt FSM

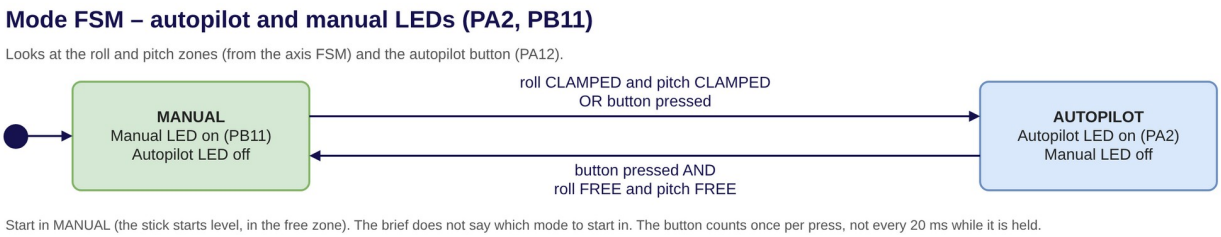


Figure 3 Mode FSM and indicator LEDs

5 Results and discussion

Table 1 lists my 18 Wokwi test cases, with angles given as roll, pitch and tilt. Cases that involve a return transition are read as sequences that start from the indicated previous state, not as independent inputs. The calculated magnitudes range from 0.999944g to 1.000053g. Case 7 starts from reset. Flattening after case 6 keeps AUTO until a new button press.

Cases 1–3 give the required minimum coverage. A 35° roll with zero pitch produces roll compliance, and the matching pitch input produces pitch compliance. With about 15° on both axes, each stays FREE while the combined tilt is about 21.2°, which activates tilt-only compliance. Roll-only compliance does not mean the tilt servo is inactive, because roll also contributes to the combined metric.

Cases 4–6 cover axis clamping and automatic autopilot entry. Cases 8, 9 and 18 cover button-controlled transitions and the restriction on returning to MANUAL. After automatic entry, flattening the input must leave AUTO selected until a fresh button press, and pressing while the axes stay clamped must keep AUTO.

Cases 10–15 cover the hysteresis bands. Once an axis has entered COMPLIANT, 27° keeps that state, whereas approaching 27° from FREE keeps FREE. Likewise 47° keeps CLAMPED after the angle has exceeded 50°. Cases 16 and 17 cover the combined tilt clamp and its return transition.

The angles in the table are rounded. The reconstructed angles can land slightly above or below a threshold. Case 16 is especially sensitive because its nominal 50° axis values lie near the clamp boundary, so the video should keep the actual readings and the preceding states. Exact boundary tests should show whether a result comes from numerical rounding or from an incorrect transition.

Table 1 Wokwi cases I tested

Case	Acceleration in g	Roll pitch tilt	Expected states and outcome
1 Roll-only COMPLIANT	(0.0000, 0.5736, 0.8192)	35.0°, 0.0°, 35.0°	R=COMP, P=FREE, T=COMP, MANUAL; pass
2 Pitch-only COMPLIANT	(-0.5736, 0.0000, 0.8192)	0.0°, 35.0°, 35.0°	R=FREE, P=COMP, T=COMP, MANUAL; pass
3 Tilt-only COMPLIANT	(-0.2588, 0.2500, 0.9330)	15.0°, 15.0°, 21.2°	R=FREE, P=FREE, T=COMP, MANUAL; pass
4 Roll → CLAMPED	(0.0000, 0.8192, 0.5736)	55.0°, 0.0°, 55.0°	R=CLAMP, P=FREE, T=COMP, MANUAL; pass
5 Pitch → CLAMPED	(-0.8192, 0.0000, 0.5736)	0.0°, 55.0°, 55.0°	R=FREE, P=CLAMP, T=COMP, MANUAL; pass
6 Both CLAMPED → AUTO	(-0.8192, 0.4698, 0.3290)	55.0°, 55.0°, 77.8°	R=CLAMP, P=CLAMP, T=CLAMP, AUTO; pass
7 Flat / both FREE	(0.0000, 0.0000, 1.0000)	0.0°, 0.0°, 0.0°	All FREE, MANUAL; pass

Case	Acceleration in g	Roll pitch tilt	Expected states and outcome
8 Button + both FREE → MANUAL	flat after case 6	0°, 0°, 0°	AUTO→MANUAL on rising edge; pass
9 Button in MANUAL → AUTO	(0,0,1) + press	0°, 0°, 0°	MANUAL→AUTO; pass
10 Deadband enter @30°	(0.0000, 0.5000, 0.8660)	30.0°, 0.0°, 30.0°	R: FREE→COMP; pass
11 Deadband hold @27°	(0.0000, 0.4540, 0.8910)	27.0°, 0.0°, 27.0°	R stays COMP (25–30 band); pass
12 Deadband leave @24°	(0.0000, 0.4067, 0.9135)	24.0°, 0.0°, 24.0°	R→FREE; T still COMP; pass
13 CLAMP enter @51°	(0.0000, 0.7771, 0.6293)	51.0°, 0.0°, 51.0°	R: COMP→CLAMP; pass
14 CLAMP hold @47°	(0.0000, 0.7314, 0.6820)	47.0°, 0.0°, 47.0°	R stays CLAMP (45–50 band); pass
15 CLAMP leave @45°	(0.0000, 0.7071, 0.7071)	45.0°, 0.0°, 45.0°	R→COMP; pass
16 Tilt CLAMPED	(-0.7660, 0.4924, 0.4132)	50.0°, 50.0°, 70.7°	T=CLAMP; axes COMP; MANUAL; pass
17 Tilt leave ≤65°	(-0.7071, 0.5000, 0.5000)	45.0°, 45.0°, 63.6°	T: CLAMP→COMP; pass
18 Button while CLAMPED	case 6 + press	55°, 55°, 77.8°	Stay AUTO; pass

6 Verification and limitations

The supplied mode test first failed to link because it had no servo_deg_to_us test stub. After that helper was supplied separately for review, all five mode checks passed. These checks cover the mode and LED cases on mocked hardware only. They do not show that the complete firmware works.

A separate host check exposed a sudden-input limitation. Starting in FREE with roll and pitch both at 60° moves the machines to COMPLIANT on the first update, commanding 150° on each axis and about 169.7° for tilt. On the next update they clamp at 140° and enter AUTO. The switch statements allow one transition per call, so gradual slider tests can pass while a large jump briefly exceeds the intended clamp. The diagrams' sequential transitions imply the same limitation.

The verification would be stronger with tests for negative-angle transitions, abrupt changes, tilt return boundaries and sensor failures. A fresh embedded build and an independent replay of the demonstration video were not completed during the review. The supplied tests therefore support the scenarios I report, but they do not show that every possible execution path is correct.

7 Lecture attribution and conclusion

I2C_Master_Init and check_timeout closely match the Lecture 8 examples. I2C_Master_ReadRegs and I2C_Master_WriteReg follow the same lecture's transaction and timeout structure [3]. PWM_Init adapts the channel-specific CCMR and CCER configuration from Lecture 5's TIM3_PWM1_Init, including compare preload and channel enables [2]. The header already acknowledges Lecture 8, but the PWM adaptation still needs a Lecture 5 source comment.

Lecture 7 supports the state-diagram and switch-case design method [4]. Lecture 9 gives context for mathematical subroutines, while the orientation equations themselves come from the assignment [1,5]. I found no active UART implementation from Lecture 6 in the application [6]. I do not treat similarity of a general method alone as proof of copied code.

The controller implements the main required sensing, feedback and mode-selection structure. The cases I report cover the required compliant examples and additional clamping and hysteresis sequences. The main remaining

limitation is the transient response to jumps across several regions. Fixing that, and completing a reproducible build and a video check, would give stronger evidence of complete correctness.

References

[1] Adelaide University. ENGE X200 Microcontroller Programming Assignment 2. 2026.

[2] Adelaide University. Lecture 5 Capture and Compare. PWM Wrapper Function and Main and ISR examples, slides 32–35.

[3] Adelaide University. Lecture 8 SPI and I2C. I2C initialization, timeout, read and write function examples.

[4] Adelaide University. Lecture 7 Finite State Machines.

[5] Adelaide University. Lecture 9 ADC.

[6] Adelaide University. Lecture 6 UART.

Source reviewed: hiten011/microProgramming, assigment2, commit 3a85950. Files: main.c, helper/helper.c, stm_helper/my_stm_helper.c, docs/Results.md and tests/test_mode_fsm.c.